

Simple Data Structures -1

Arrays, Stacks and Queues

Y.N. Srikant

Visiting Professor
Chanakya University
(Formerly) Professor
Computer Science and Automation
Indian Institute of Science
Bangalore 560 012

Abstract Data Type and Data Structure

- Abstract data type (ADT)
 - A mathematical model with a collection of operations defined on the data inside that model.
 - Operations have well defined properties and affect only local data inside the ADT model.
 - The data in the model can be accessed only with the operations defined in the ADT.
 - ADT can be used to prove properties of an implementation.
 - In practice, they guide implementations.
- Data Structure
 - A collection of variables of different types connected in various ways.
 - Used to represent the data part of the mathematical model of an ADT.
 - Operations and the data structure together form an implementation of an ADT.

Array ADT

- 1 An array is a set of pairs of the form (index, value).
- 2 No two pairs of this set have the same index.
- 3 The operations supported on an array are the following.
 - CREATE(A) - creates an empty array A.
 - STORE(A,i,x) - Adds the pair (i,x) to A. Overrides the old pair if one already exists.
 - RETRIEVE(A,i) - Retrieves and returns the value at index i in array A.
- 4 An array is usually a primitive data type in most programming languages.
- 5 Each value in an array can itself be an array.
- 6 Arrays are usually of fixed size.
- 7 Arrays are used extensively in numerical computation to store and manipulate matrices and tensors (higher dimensional matrices).

Array Mapping in C and Numpy - 1

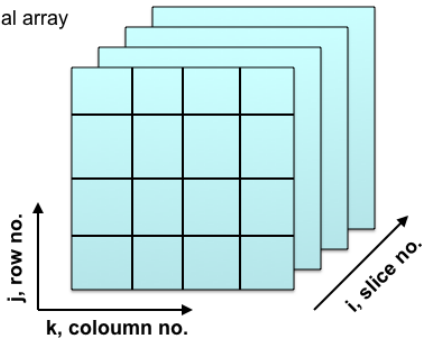
Arrays in Numpy and C are of fixed size.

Assume that $A[n_1, n_2, n_3]$ is a 3-D array of integers and that $B[n_1 * n_2 * n_3 * 4]$ is a 1-D array of bytes, both of the same capacity. B corresponds to memory. Each integer is assumed to take 4 bytes. Assume row-major ordering.

- Size of each slice is $n_2 * n_3 * 4$ bytes, size of each row is $n_3 * 4$ bytes and size of each element is 4 bytes.
- $A[i, j, k]$ is mapped to the address $B[i * n_2 * n_3 * 4 + j * n_3 * 4 + k * 4]$.
- For efficient computation, this can be written as $B[(((i * n_2) + j) * n_3 + k) * 4]$

Array Mapping in C and Numpy - 2

3-dimensional array



1-D representation of 3-D array

$\text{offset} = (((i * n_2) + j) * n_3) + k) * \text{ele_size}$



Homework: Not to be submitted

Find an efficient mapping to a 1-D array for the following types of matrices.

- 1 A diagonal 2-D matrix $a[n, n]$ (all elements other than the diagonal are zero). 1-D array should not use more than n elements.
- 2 Lower diagonal 2-D matrix ($a[i, j] = 0$ for $i = 0, \dots, n - 1$, and $j = i + 1, \dots, n - 1$). 1-D array should not use more than $(n^2 + n)/2$ elements.
- 3 Similarly, consider upper diagonal, and band matrices.

Tensor Mapping in TensorFlow - An Example

- Video data is stored as a 5-dimensional tensor in TensorFlow - (*batch, frames, height, width, channels*).
 - *Batch(B)*: Number of videos to be processed.
 - *Frames(F)*: Number of frames in each video. Each frame is essentially an image frozen in time.
 - *Height(H), Width(W)*: Each frame has *height x width* number of pixels (i.e., *rows x columns*).
 - *Channels(C)*: The number of colors in each pixel. Usually, each color requires one byte or one float (4 bytes). For example, for RGB (#channels = 3), each of R,G, and B require one byte to store the intensity.
- *Embeddings* used in deep learning are also stored as multi-dimensional tensors with hundreds of dimensions.
- Embeddings are stored in *vector databases*, with each vector being a multi-dimensional tensor (with 100's or 1000's of dimensions).
- Fast access of vectors is essential.

Embeddings

- Embeddings are numerical representations of data.
- They convert complex, high-dimensional data into low-dimensional vectors.
- This transformation allows machines to process and analyze data efficiently.
- Embeddings are usually obtained using deep learning networks.
- Example 1: words in a language can be represented as vectors of real numbers, where semantically similar words are positioned closely together.
- Example 2: images can be represented as vectors, and similar images will be placed closer. For example, images of cat and kitten will be closer than the images of cat and dog.

Homework: Study embeddings in detail.

Tensor Mapping in TensorFlow - An Example

- This *meta data*, a 5-tuple of the form (B,F,H,W,C), (example - (5,10,8,10,3)), is stored separately from the raw data.
- The raw data of a video tensor is stored as one large contiguous memory buffer (see next slide).
- Batches are slices of this buffer, frames are slices of each batch, rows are slices of each frame, etc.

Homework: For a video tensor access $vt[b,f,h,w,c]$, where, b , f , h , w , and c are the batch number, frame number, row number, column number, and color number (assume $R=0$, $G=1$, and $B=2$), write down the mapping function just like in the array mapping case. Assume that each color in a pixel requires 4 bytes (instead of one byte that was mentioned earlier).

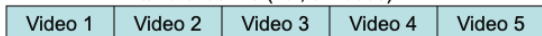
Video Tensor Mapping in TensorFlow

Total storage for the video tensor = $5 \times 10 \times 8 \times 10 \times 3 \times 1 = 12000$ bytes

Storage requirement

Per batch 2400 bytes

#Batches = 5 (i.e., 5 videos)



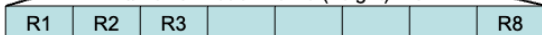
Per frame 240 bytes

#Frames in each video = 10



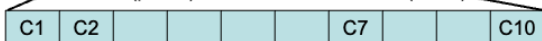
Per row 30 bytes

#Rows in each frame (height) = 8

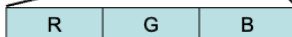


Per column
(i.e., pixel) 3 bytes

#Columns (pixels) in each row of a frame (width) = 10



Per channel (i.e., color) 1 byte

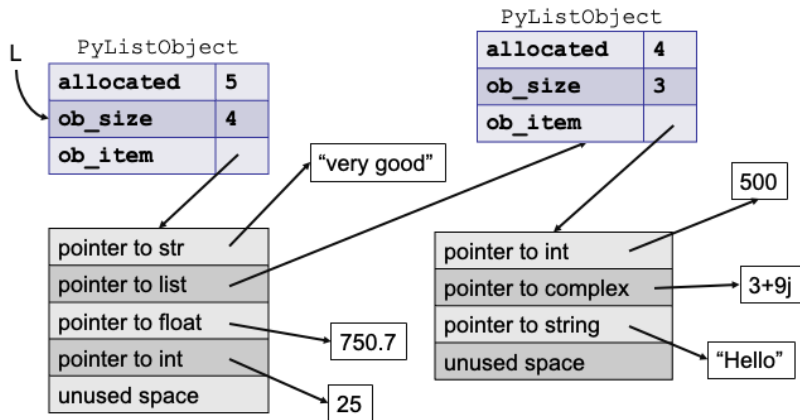


#Colors (channels) per pixel = 3 (i.e., 3 bytes/pixel)

List Mapping in Python

Lists in Python can change size dynamically.

```
L = ["very good", [500, 3+9j, "Hello"], 750.7, 25]
```

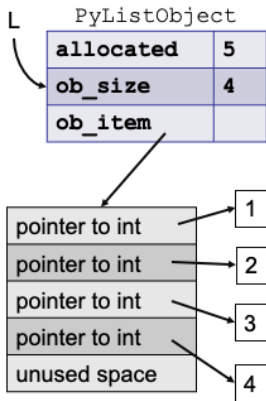


`PyListObject` is a C-struct and `ob_item` is an array of C-pointers.

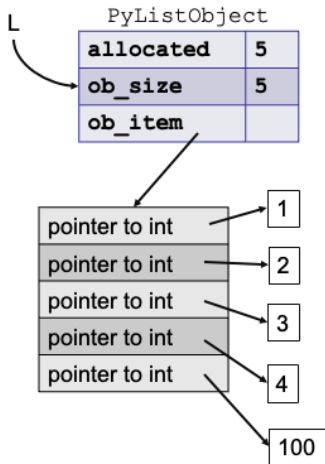
List Mapping in Python

Lists in Python can change size dynamically.

`L = [1,2,3,4]`



After `L.append(100)`



List Mapping in Python

After L.append(200), L.append(300), L.append(400)

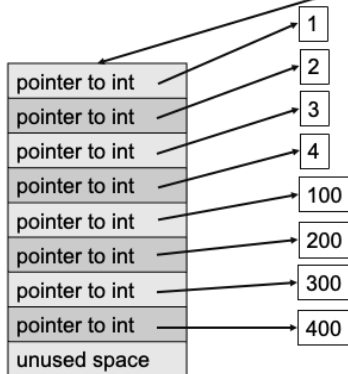
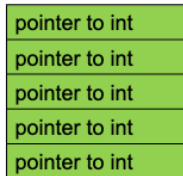
Once available space in the `ob_item` array is exhausted, a new array of larger size is allocated and all the items from the old array are copied to the new one. The old array is freed.

L

PyListObject

allocated	10
ob_size	8
ob_item	

Garbage for collection



Sparse Matrix and Tensor Representations

- 1 Coordinate format (COO): Used for tensors in TensorFlow. Generally not used for matrices.
- 2 Compressed Sparse Row format (CSR): Most commonly used format for matrices and internally in TensorFlow.
- 3 Compressed Sparse Column format (CSC): Popular in scripting languages. It is the default format for sparse matrices in the languages *Julia* and *Matlab*.

We discuss COO and CSR formats. CSC is similar to CSR and is not discussed (for self study).

Homework: Write Python programs to multiply two matrices in COO and CSR formats.

Sparse Coordinate (COO) Format

$$A = \begin{bmatrix} 14 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 50 & 0 & 17 \\ 12 & 6 & 2 & 0 \end{bmatrix}$$

COO Format:

(value, row index, column index)

Each of these three items is stored as an array. TensorFlow stores each (row index, col index) as a tuple.

value = [6 50 14 17 2 10 12]

row_index = [3 2 0 2 3 0 3]

col_index = [1 1 0 3 2 2 0]

For example, 17 is $A[2,3]$, and 6 is $A[3,1]$. Row 1 with all 0's does not appear in any array.

Sparse COO-Sorted Format

$$A = \begin{bmatrix} 14 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 50 & 0 & 17 \\ 12 & 6 & 2 & 0 \end{bmatrix}$$

COO Format with sorting:

(value, row index, column index)

We could sort entries by row and then by column.

value = [14 10 50 17 12 6 2]

row_index = [0 0 2 2 3 3 3]

col_index = [0 2 1 3 0 1 2]

For example, values 14 and 10 are both in row 0 of A. Row 1 with all 0's does not appear in any array.

Compressed Sparse Row (CSR) Format

$$A = \begin{bmatrix} 14 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 50 & 0 & 17 \\ 12 & 6 & 2 & 0 \end{bmatrix}$$

$value = [14 \quad 10 \quad 50 \quad 17 \quad 12 \quad 6 \quad 2]$
(size = #non-zero values in A)

$row_ptr = [0 \quad 2 \quad 2 \quad 4 \quad 7]$
(size = #rows of A + 1)

$col_index = [0 \quad 2 \quad 1 \quad 3 \quad 0 \quad 1 \quad 2]$
(size = # non-zero values in A)

CSR Format: (value, row pointer, column index)

Indices in all the three arrays and matrix A above begin from 0.
For example, A[2,1] is 50 and A[0,0] is 14.

- *row_ptr* tells us where in the *value* array
 - a particular row begins.
- For example, 4 as the third entry of the *row_ptr* array
 - says that row 3 begins
 - at index 4 in the *value* array.
- *col_index* gives the column number of the non-zero value.

Compressed Sparse Row (CSR) Format

$$A = \begin{bmatrix} 14 & 0 & 10 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 50 & 0 & 17 \\ 12 & 6 & 2 & 0 \end{bmatrix}$$

$value = [14 \quad 10 \quad 50 \quad 17 \quad 12 \quad 6 \quad 2]$
(size = #non-zero values in A)

$row_ptr = [0 \quad 2 \quad 2 \quad 4 \quad 7]$
(size = #rows of A + 1)

$col_index = [0 \quad 2 \quad 1 \quad 3 \quad 0 \quad 1 \quad 2]$
(size = # non-zero values in A)

- $n = row_ptr[i+1] - row_ptr[i]$ gives
 - the number of nonzero entries in row i of A, and hence
 - the number of entries for row i of A, in the *value* array,
 - from $row_ptr[i]$ to $row_ptr[i]+n-1$.
 - *col_index* gives the corresponding column numbers.
- For example, $row_ptr[4] - row_ptr[3] = 7 - 4 = 3$
 - says that row 3 of A has 3 nonzero elements
 - and 3 entries in the *value* array, $value[4]$ ($= A[3,0] = 12$),
 - $value[5]$ ($= A[3,1] = 6$), and $value[6]$ ($= A[3,2] = 2$).
- Row 2 of A having all zero elements is reflected
 - in $row_ptr[1]$ and $row_ptr[2]$ having the same value (2)
 - and thus $\#elements = 0$.

Stack ADT

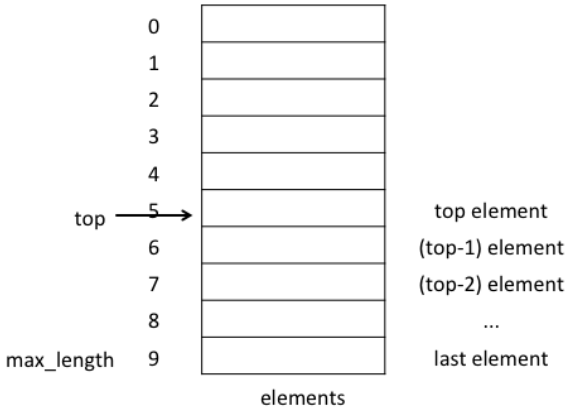
A stack is a *last-in-first-out* structure.

A stack ADT is a collection of elements with the following operations:

- 1 void MAKENULL(S) - Make an empty stack, S
- 2 eletype TOP(S) - Return the top element of S
- 3 eletype POP(S) - Return and delete the top element of S
- 4 void PUSH(x,S) - Insert the element x at the top of S
- 5 boolean EMPTY(S) - True if S is empty, else false

Why is a `is_FULL(S)` operation not defined? That is because abstract stacks never become full! But, implementations must deal with this issue.

An Array Implementation of a Stack



Stacks in Python -1

```
class Stack:
    def __init__(self):
        self.items = []
    def Empty(self):
        return self.items==[]
    def push(self, item):
        self.items.append(item)
    def pop(self):
        return self.items.pop()
    def top(self):
        return self.items[len(self.items)-1]
```

Stacks in Python -2

```
class Stack:
    def __init__(self):
        self.items = []

    def Empty(self):
        return self.items == []

    def push(self, item):
        self.items.insert(0,item)

    def pop(self):
        return self.items.pop(0)

    def top(self):
        return self.items[0]
```

|

| s = Stack()

| s.push('Hello!')

| s.push('How are

| you?')

| print(s.pop())

|

| Push() and Pop()

| both require $O(n)$

| time!

|

Homework: Not to be submitted

Write a program in Python to implement two stacks in a single list. Try to utilize the space efficiently. Can you generalize your technique to several stacks? What are the limitations of this approach?

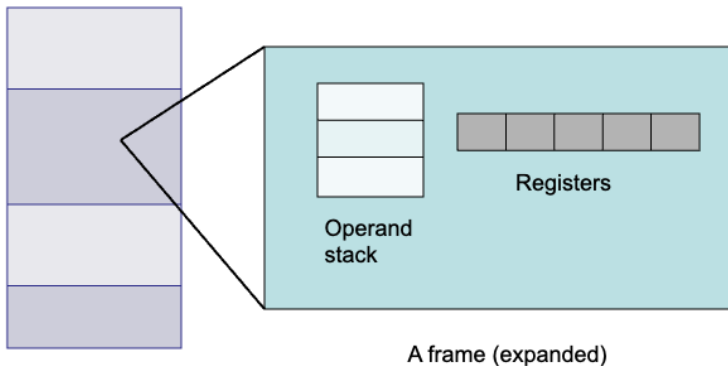
Applications of Stacks

- ① Support for function calls with recursion.
 - Runtime memory management (activation records).
 - Storing return addresses.
- ② Stack machine implementation.
 - Java Virtual Machine (JVM), Python Virtual Machine.
- ③ Postfix, prefix, and infix expression evaluation.

Basics of the Java Virtual Machine (JVM) - 1

- Java *bytecode* is the instruction set of the JVM.
- Each instruction consists of one byte of opcode and zero or more bytes of operands.
- Java bytecode is designed to run on most common platforms, such as desktops, laptops, phones, tablets, etc.
- The bytecode comprises data manipulation, control transfer, object creation and manipulation, and method invocation instructions.
- The frame (activation record) of a method call in JVM has both an *operand stack* and a set of *registers*.
 - *Operand stack*: used for computations, such as addition, multiplication, etc.
 - *Registers*: used to store local variables and method arguments.

JVM Frames



Stack of
frames of
method calls

Operand stack is used for computations and registers are used to store local variables and method arguments.

Basics of the Java Virtual Machine (JVM) - 2

Java program

```
x = 10;  
x = 25-x;  
c = x * 80;
```

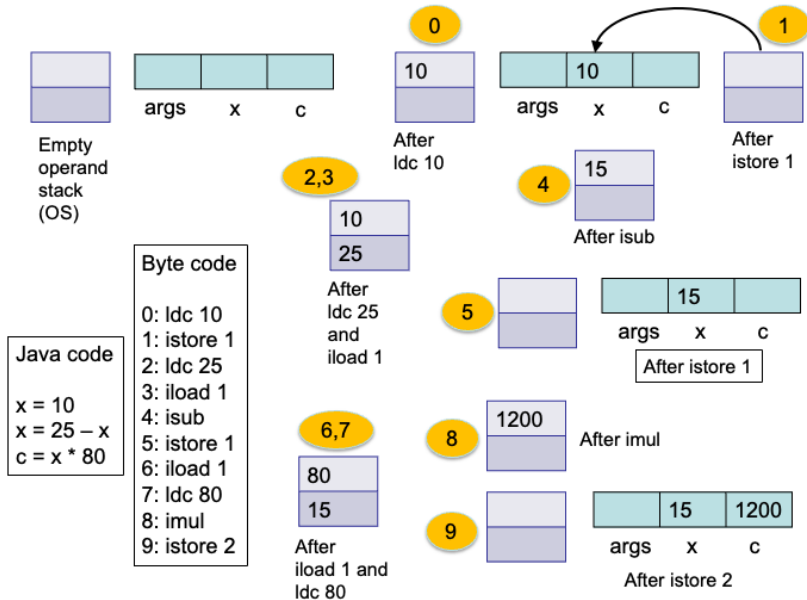
Byte code:

```
stack=2, locals=2
```

```
0: ldc 10  
1: istore 1  
2: ldc 25  
3: iload 1  
4: isub  
5: istore 1  
6: iload 1  
7: ldc 80  
8: imul  
9: istore 2  
10: return
```

- Java byte code defines several types of “load constant”, “load”, “store”, arithmetic, and “branch” instructions.
- For example, `iconst_<i>`, `dconst_<d>`, `fconst_<f>`, `bipush`, `sipush`, and `ldc` are all different types of “load constant” instructions.
- We consider only `ldc`, “load constant” instruction in the example shown beside.

JVM Byte Code Execution



Postfix and Prefix Expression Evaluation

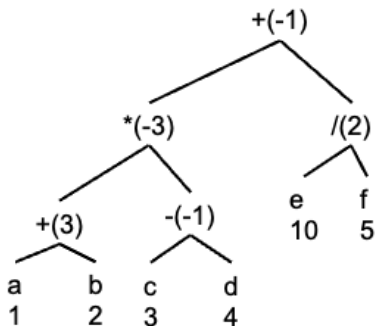
Postfix expression evaluation

- Example: $ab+cd-*ef/+$ and $1\ 2\ +\ 3\ 4\ -\ *\ 10\ 5\ /\ +\ (= -1)$.
 - 1 Scan the input from left to right.
 - 2 Push operands into a stack until an operator is met.
 - 3 Pop the required number of operands for that operator from the stack.
 - 4 Evaluate the operator and push the result into the stack.
 - 5 Repeat this procedure until the input is empty.
 - 6 The stack at this point must have just the final result.

Prefix Expression Evaluation

- Example: $+*+ab-cd/ef$ and $+*\ +\ 1\ 2\ -\ 3\ 4\ /\ 10\ 5\ (= -1)$.
- Scan the input from RIGHT to LEFT. Do exactly as in postfix evaluation.

Infix Expression Evaluation See second slide from the current slide (self reading).



Postfix expression
ab+cd-*ef/+

1 2 + 3 4 - * 10 5 / +

Prefix expression
+*+ab-cd/ef

+ * + 1 2 - 3 4 / 10 5

Infix expression
(a+b)*(c-d)+(e/f)

(1+2)*(3-4)+(10/5)

Note that the order of evaluation of subexpressions in prefix and postfix expressions may be different.

1. While there are still tokens to be read in,
 - 1.1 Get the next token.
 - 1.2 If the token is:
 - 1.2.1 A number: push it onto the value stack.
 - 1.2.2 A variable: get its value, and push onto the value stack.
 - 1.2.3 A left parenthesis: push it onto the operator stack.
 - 1.2.4 A right parenthesis:
 - 1 While the thing on top of the operator stack is not a left parenthesis,
 - 1 Pop the operator from the operator stack.
 - 2 Pop the value stack twice, getting two operands.
 - 3 Apply the operator to the operands, in the correct order.
 - 4 Push the result onto the value stack.
 - 2 Pop the left parenthesis from the operator stack, and discard it.
 - 1.2.5 An operator (call it thisOp):
 - 1 While the operator stack is not empty, and the top thing on the operator stack has the same or greater precedence as thisOp,
 - 1 Pop the operator from the operator stack.
 - 2 Pop the value stack twice, getting two operands.
 - 3 Apply the operator to the operands, in the correct order.
 - 4 Push the result onto the value stack.
 - 2 Push thisOp onto the operator stack.
2. While the operator stack is not empty,
 - 1 Pop the operator from the operator stack.
 - 2 Pop the value stack twice, getting two operands.
 - 3 Apply the operator to the operands, in the correct order.
 - 4 Push the result onto the value stack.
3. At this point the operator stack should be empty, and the value stack should have only one value in it, which is the final result.

Homework: Not to be submitted

- 1 Write programs in Python to perform infix and postfix expression evaluations. Consider as many operators as you can.
- 2 Write a program in Python to convert an infix expression to postfix.

Micro-project -1

- 1 Implement a simulator for a small and simple subset of Java byte code in Python. Assume maximum sizes of the operand stack and register set (local variables) to be 10 each (you may use a larger size, if you need it).

The subset of the instruction set that you need to simulate consists of `{ldc, iload, istore, iadd, isub, imul, idiv, ifeq, iflt, ifgt}`, and two special instructions, `{read, print}`, that are not available in Java byte code. `read` reads an integer from the keyboard, and pushes it onto the operand stack. Similarly, `print` prints the integer on the top of the stack on the console. You need to implement the `ldc` instruction for integers only.

Supply a set of non-trivial programs in this subset of Java byte code that you have tested the simulator with. You may learn details of the byte codes from the URL:
<https://docs.oracle.com/javase/specs/jvms/se11/html/jvms-6.html>

Queue ADT

A Queue is a FIRST-IN-FIRST-OUT structure.

A Queue is a collection of elements with the following operations:

- ❶ **MAKENULL(Q)** - Makes a null queue, Q
- ❷ **FRONT(Q)** - Returns first element of Q
- ❸ **ENQUEUE(x,Q)** - Inserts x at the end of Q
- ❹ **DEQUEUE(Q)** - Deletes first element of Q
 - Another version: **DEQUEUE(Q,x)** - Deletes first element of Q and assigns it to x
- ❺ **EMPTY(Q)** - True if Q is empty, else false

A flat array is not a good candidate for implementing a queue.

Queues in Python

```
class Queue:
    def __init__(self):
        self.items = []

    def Empty(self):
        return self.items == []

    def enqueue(self, item):
        self.items.insert(0,item)

    def dequeue(self):
        return self.items.pop()

q=Queue()
q.enqueue(15)
q.enqueue('Welcome!')
print(q.dequeue())
```

Enqueue requires $O(n)$ time and Dequeue $O(1)$ time!

Applications of Queues

- 1 Operating systems use print queues and task queues.
- 2 Railway reservation systems use queues to issue tickets.
- 3 Airport simulation.
- 4 In breadth-first search algorithm on graphs.
- 5 Image component labelling.

Image Component Labeling

- Before labeling, $\text{pixel}[r,c] = 0$ or 1 .
- Orange part is the boundary of 0 valued pixels (not part of the image).
- After labeling components, pixels in a component get values $2, 3, 4, \dots$.
- These are the component numbers.

0	0	0	0	0	0	0	0	0	0
0			1	1					0
0				1					0
0					1				0
0				1	1	1			0
0					1				0
0		1	1	1			1	1	0
0		1	1		1	1	1	1	0
0			1				1		0
0	0	0	0	0	0	0	0	0	0

0	0	0	0	0	0	0	0	0	0
0			2	2					0
0				2					0
0					3				0
0				3	3	3			0
0					3				0
0		4	4	4			5	5	0
0		4	4		5	5	5	5	0
0			4				5		0
0	0	0	0	0	0	0	0	0	0

Image Component Labeling

```
for each row r do { //Component id=1 initially
    for each column c in row r do {
        if (pixel[r,c]==1) //new component id
        { pixel[r,c]=++id; HERE.row=r; HERE.col=c;
            do { for each neighbor NBR of HERE do {
                //in up,down,left,right directions
                //by one pixel only
                if (pixel[NBR.row,NBR.col]==1) {
                    pixel[NBR.row,NBR.col]=id;
                    enqueue(Q,NBR);
                }
            }
            if(empty(Q)) break; HERE=dequeue(Q);
        } while(TRUE);
    }
}
```

Time complexity is $O(\text{\#rows} * \text{\#cols})$

Image Component Labeling in Python: Results

Input image

```
-----  
0 0 0 0 0 0 0 0 0 0  
0 0 0 1 1 0 0 1 1 0  
0 0 0 0 1 0 1 0 1 0  
0 0 0 0 0 1 0 1 0 0  
0 0 0 0 1 1 1 0 0 0  
0 0 0 0 0 1 0 0 0 0  
0 0 1 1 1 0 0 1 1 0  
0 0 1 1 0 1 1 1 1 0  
0 0 0 1 0 0 0 1 0 0  
0 0 0 0 0 0 0 0 0 0
```

Output with comp labels

```
-----  
0 0 0 0 0 0 0 0 0 0  
0 0 0 2 2 0 0 3 3 0  
0 0 0 0 2 0 4 0 3 0  
0 0 0 0 0 5 0 6 0 0  
0 0 0 0 5 5 5 0 0 0  
0 0 0 0 0 5 0 0 0 0  
0 0 7 7 7 0 0 8 8 0  
0 0 7 7 0 8 8 8 8 0  
0 0 0 7 0 0 0 8 0 0  
0 0 0 0 0 0 0 0 0 0
```

Double Ended Queue (Deque)

A Deque is a generalized version of stack and queue. Deques are already available in Python. It is a collection of elements with the following operations:

- 1 **MAKENULL(Q)** - Makes a null deque, Q ($Q = \text{deque}()$).
- 2 **ENQUEFRONT(x,Q)** - Inserts x at the front of Q ($Q.appendleft()$).
- 3 **DEQUEFRONT(Q)** - Deletes and returns the element at the front of Q ($Q.popleft()$).
- 4 **ENQUEREAR(x,Q)** - Inserts x at the rear of Q ($Q.append()$).
- 5 **DEQUEREAR(Q)** - Deletes and returns the element at the rear of Q ($Q.pop()$).
- 6 **EMPTY(Q)** - True if Q is empty, else false (not directly available).
- 7 **SIZE(Q)** - Returns the number of elements in Q ($\text{len}(Q)$).

Python deques are very efficient. All enqueue and dequeue operations require only $O(1)$ time!

Using Deques in Python

```
from collections
    import deque
dq=deque()
print(dq)
print("Enqueue
    from the rear")
dq.append(4)
print(dq)
dq.append('dog')
print(dq)
print("Enqueue
    from the front")
dq.appendleft(True)
print(dq)
```

```
dq.appendleft(78.54)
print(dq)
print("Dequeue
    from the front")
dq.popleft()
print(dq)
dq.popleft()
print(dq)
print("Dequeue
    from the rear")
while len(dq) > 0:
    dq.pop()
    print(dq)
```

Deque: Results

```
deque([])  
Enqueue from the rear  
deque([4])  
deque([4, 'dog'])  
Enqueue from the front  
deque([True, 4, 'dog'])  
deque([78.54, True, 4, 'dog'])  
Deque from the front  
deque([True, 4, 'dog'])  
deque([4, 'dog'])  
Deque from the rear  
deque([4])  
deque([])
```

Homework (Not for Submission)

- 1 Show how to implement stacks and queues using deques in Python.
- 2 Implement a palindrome checker in Python using a deque.
- 3 Implement a fixed size deque of strings in Python, which automatically removes an item from the front or rear, when the number of elements in the deque is exceeded during enqueue operations. The new item now replaces the removed item.